Vapor Stone
RAIL SYSTEMS
A Wabtec company

10 655 Henri-Bourassa Blvd., West
Saint-Laurent (Québec) Canada
H4S 1A1

Tel: 514.335.4200
Fax: 514.335.4231

www.wabtec.com

Regional Transportation District

RTD Denver EMU

Project # 7230

Door System

Neuron Gateway Application Software Design Description (SDD)

Contract Number		CDRL Number	
		Customer Ref. Number	-

NOTICE
Plans, specifications and engineering data submitted herewith or which may be submitted, remain the property of **Vapor Stone Rail Systems** and shall not be reproduced or altered or used in whole or in part except by express consent of **VSRS**.

AVIS
Les plans, spécifications et données techniques soumis ci-joints, ou qui peuvent être soumis, restent la propriété de **Vapor Stone Rail Systems** et ne doivent pas être reproduits, altérés ou utilisés, totalement ou partiellement, sans le consentement express de **VSRS**.

	Name / Title	Signature / Date
Prepared :	James Kim Software Designer	James Kim 2011-10-17
Verified :	Martin Ouellet, Eng. Software Department Supervisor	2011-10-17
Approved :	Aldo Di Rocco Project Manager	2011-10-17

Document No.	**201111459**	Revision	**1**
		Date (yyyy-mm-dd)	**2011-10-14**

Revision List

Rev.	ECD#	Date	By	Description of changes
1	---	2011-10-14	James Kim	Official release.
0.1	---	2011-10-13	James Kim	Implementing comments from internal inspection I-1860.
0	---	2011-10-04	James Kim	Original issue.

Preface

This Software Design Description (SDD) complies with the requirements of the IEEE 1016-1998 and IEEE 1558-2004.

Table of Contents

Revision List ... 2
Preface ... 3
Table of Contents ... 4
List of Figures ... 6
List of Tables ... 7

1 INTRODUCTION ... 8

1.1 Purpose ... 8
1.2 Scope ... 8
1.3 Definitions and Acronyms ... 8
1.3.1 Definitions ... 8
1.3.2 Acronyms ... 8
1.4 Document Organization ... 10

2 REFERENCES ... 11

3 SOFTWARE DESIGN ... 12

3.1 Hardware Block Diagram ... 12
3.2 Decomposition Description ... 13
3.2.1 Entities Decomposition ... 13
3.2.1.1 Neuron Firmware ... 13
3.2.1.2 Initialization Module ... 14
3.2.1.3 Self Test Module ... 14
3.2.1.4 Main Process Module ... 14
3.2.1.5 SPI Module ... 14
3.2.1.6 Status Process Module ... 14
3.2.1.7 Time Update Module ... 14
3.2.1.8 SCI Update ... 14
3.2.1.9 FTP Module ... 14
3.2.1.10 Version Module ... 14
3.2.2 Data Decomposition ... 15
3.3 Dependency Description ... 16
3.3.1 Inter-Entity Dependencies ... 16
3.3.1.1 Initialization Module ... 16
3.3.1.2 Door Location Hardware I/O ... 16
3.3.1.3 SPI Module ... 16
3.3.1.4 DCU Data ... 17
3.3.1.5 SPI Interface ... 17
3.3.1.6 Status Process Module ... 17
3.3.1.7 Time Update Module ... 17
3.3.1.8 Version Module ... 17
3.3.1.9 SCI Update Module ... 17
3.3.1.10 FTP Module ... 17
3.3.1.11 Car Network ... 17
3.3.2 Inter-Process Dependencies ... 18
3.3.2.1 Initialization ... 18
3.3.2.2 Door Location Hardware I/O ... 18
3.3.2.3 SPI ... 18
3.3.2.4 DCU Data ... 18
3.3.2.5 SPI Interface ... 19
3.3.2.6 Status Process ... 19
3.3.2.7 Time Update ... 19
3.3.2.8 SCI Update and FTP ... 19
3.3.2.9 Version ... 19
3.3.2.10 Car Network ... 19

3.3.3 Data Dependencies ... 19
3.4 Interface Description ... 20
3.4.1 Internal Interfaces ... 20
3.4.1.1 Main Process Module ... 20
3.4.1.2 Initialization Module ... 20
3.4.1.3 Self Test Module ... 21
3.4.1.4 Status Process Module ... 22
3.4.1.5 Time Update Module ... 22
3.4.1.6 FTP Module ... 22
3.4.1.7 Version Module ... 24
3.4.1.8 SPI Module ... 24
3.4.2 External Interfaces ... 26
3.4.2.1 Door Location Hardware I/O ... 26
3.4.2.2 SPI Interface ... 26
3.4.2.3 Car Network ... 26
3.4.3 Data Dictionary ... 27
3.5 Detailed Design ... 28
3.5.1 Entities Detailed Design ... 28
3.5.1.1 Initialization Module ... 28
3.5.1.2 Main Process Module ... 32
3.5.1.3 Self Test Module ... 32
3.5.1.4 SPI Module ... 32
3.5.1.5 SCI Update Module ... 38
3.5.1.6 Status Process Module ... 38
3.5.1.7 Network Health Module ... 39
3.5.1.8 Time Update Module ... 39
3.5.1.9 FTP Module ... 39
3.5.1.10 Version Module ... 42
3.5.2 Data Entities Detailed Design ... 43
3.5.2.1 Network Variables ... 43
3.5.2.2 SPI Communication Buffer ... 43
3.5.2.3 SCI Versions Structure ... 43
3.5.2.4 DCU Signals ... 43
3.5.2.5 Network Health Data ... 45
3.5.2.6 SPI Health Data ... 45
3.5.2.7 Location Data ... 45
3.5.2.8 SCI CRC Data ... 46
3.5.2.9 FTP Status Data ... 46

4 OPERATING SYSTEM SERVICES ... 47

5 ERROR / WARNING HANDLING ... 48

6 MEMORY LAYOUT ... 49

List of Figures

Figure 3-1: Hardware Block Diagram 12
Figure 3-2: Hierarchical Diagram of NGA 13
Figure 3-3: Data Decomposition of DCU Application 15
Figure 3-4: NGA Data Flow Diagram 16
Figure 3-5: Neuron Gateway Control Flow Diagram 18
Figure 3-6: initInitialization Routine Flowchart 28
Figure 3-7: when(timer_expires(MainLoopTimer)) Flowchart 32
Figure 3-8: spiDataExchange Flowchart 34
Figure 3-9: spiNeuronBufferPrepare Flowchart 34
Figure 3-10: spiIncomingDataCheck Flowchart 35
Figure 3-11: spiUpdateDCUStructs Flowchart 36
Figure 3-12: spiUpdateSCIVersionsStructs Flowchart 37
Figure 3-13: Update SCI Flowchart 38
Figure 3-14: Time Update Flowchart 39
Figure 3-15: File Transfer Architecture 40
Figure 3-16: File Transfer – Window Successfully Received 40
Figure 3-17: File Transfer – Window With Retransmission 41
Figure 3-18: when (msg_arrives(SW_VERSION_REQUEST)) Flowchart 42
Figure 6-1: DCU Neuron Memory Map 49

List of Tables

Table 3-1: DCU Location 26
Table 3-2: Network Variable List 26
Table 3-3: Neuron Configuration Structure 29
Table 3-4: Neuron Domain Table 30
Table 3-5: Neuron Address Table 30
Table 3-6: Network Variables List Table 31
Table 5-1: Faults List 48

1 INTRODUCTION

This document is the Software Design Description of the Neuron Gateway Application (NGA) developed by Vapor Stone Rail Systems.

The Neuron Gateway Application is part of the Microprocessor Based System (MBS). It is responsible for handling network communication.

1.1 Purpose

The purpose of this Software Design Description (SDD) is to describe the software implementation of the Neuron Gateway Application.

The intended audience is the Vapor Stone Rail Systems software development team and maintenance team members of the transit authority and the car builder.

1.2 Scope

The SD3 Neuron Gateway Application is a software program that runs on the Neuron™ processor. It reports the status and health of the Door Control Unit (DCU) to the car network. The NGA also receives messages from the car network.

Only the software design is detailed in this document. The target hardware is proprietary, owned and designed by Vapor Stone Rail Systems. The Car Network uses LonWorks™ product and technology and is LonTalk™ protocol compliant.

1.3 Definitions and Acronyms

1.3.1 Definitions

Door Control Panel: A panel installed in each cab, one per train side, used to transmit the train operator door commands to the train lines.

Door Control Unit: A specific processor based electronic module running a Door Control Unit Application. It is used for controlling and monitoring the door.

LonWorks: Registered trademark of Echelon corp., which presents Echelon distributed network. See http://www.echelon.com

1.3.2 Acronyms

Acronym	Meaning
ASCII	American Standard Code for Information Interchange
CBTC	Communication-Based Train Control
CD	Collision Detection
CDP	Central Diagnostic Panel
CDT	Central Diagnostic Terminal
CFG	Configuration
CMC	Communications Controller
CMN	Car Monitoring Network
CN	Car Network
CPLD	Complex Programmable Logic Device
CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
DCP	Door Control Panel
DCU	Door Control Unit
ECD	Engineering Change Document

Acronym	Meaning
EDF	Event Description File
EEPROM	Electrically Erasable/Programmable Read-Only Memory
FTP	File Transfer Protocol
I/O	Inputs/Outputs
ICD	Interface Control Document
ID	Identification
IEEE	The Institute of Electrical and Electronics Engineers
LED	Light Emitting Diode
LSB	Least Significant Bit
MBS	Microprocessor Based System
MDC	Master Door Controller
MDS	Monitoring and Diagnostic System
MSB	Most Significant Bit
N.C.	Not Connected
NG	No Good
NGA	Neuron Gateway Application
NLC	No Lan Communication
NVSRAM	Non-Volatile Static Random Access Memory
POST	Power-On Self-Test
QT	Quester Tangent
RAM	Random Access Memory
Rev	Revision
RQ	Request
RSP	Response
RTD	Regional Transportation District
Rx	Reception
SCI	Software Configuration Item
SCPT	Standard Configuration Property Type
SDD	Software Design Description
SFD	System Functional Description
SFPT	Standard Functional Profile Type
SLDT	System Level Dynamic Tester
SNVT	Standard Network Variable Type
SPI	Serial Peripheral Interface
SQAP	Software Quality Assurance Plan
SRAM	Static RAM
SRS	System Requirements Specification
Std	Standard
SVVP	Software Verification and Validation Plan
TL	Train Line
TS	Technical Specifications
Tx	Transmission
UNACKD	Unacklowledge
UNVT	User-Defined Network Variable Types
UTC	Coordinated Universal Time

1.4 Document Organization

This SDD is divided into 6 sections and the following specific information is provided within each one:

- Section 1, Introduction, states the purpose, identifies the scope, gives definitions, abbreviations and an overview of the SDD.
- Section 2, References, lists the factors that affect the product and its requirements.
- Section 3, Software Design, summarizes followings:
    - Decomposition Description: describes functions of software entities.
    - Dependency Description: gives a brief description of the dependency between each entity.
    - Interface Description: describes the interface of the entity.
    - Detailed Design: details the functions implemented by each software entity, presents and explains tasks performed by each software entity in functional conditions.
- Section 4, Operating System Services, mentions the services used in the Real-Time Operating System.
- Section 5, Error / Warning Handling, enumerates the NGA locally detected faults.
- Section 6, Memory Layout, shows Flash and RAM memory mapping.

Within the document, diagrams, flowcharts and tables are used to present the software design:

- Decision flowcharts give a graphical view of how entities and functions process work.
- The diagram Figure 3-4 describes the dependency between different modules.
- Tables contain data structures used by the Neuron Gateway Application. They also present C code.

2 REFERENCES

This section lists the number, title, revision, date, and source of all documents referenced in this SDD.

Standards

IEEE Std 1016-1998	IEEE Recommended Practice for Software Design Descriptions, September 23rd 1998, IEEE, www.ieee.org
IEEE Std 1558-2004	IEEE Recommended Practice for Software Documentation for Rail Equipment and Systems, March 23rd 2005, IEEE, www.ieee.org

Vapor Stone Rail Systems Documents

201111185	System Functional Description, RTD Denver EMU Door System, Rev. 4
201111256	Software Requirement Specification, Neuron Gateway Application, Rev. 1,
201111207	Software Project Management Plan, Door System, Rev. 0
201111208	Software Quality Assurance Plan, Door System, Rev. 1
201111186	Software Configuration Management Plan, Door System, Rev. 2
201111254	Software Verification and Validation Plan, Door System, Rev. 0
201111586	Interface Control Document – Neuron Serial Peripheral Interface, Rev. 1
S17-07	Software Design Description Template, Rev. 0, 2002-01-15

Quester Tangent Documents

DDN-0432-0080	MDS Car Monitoring Network Interface Control Document, Rev. 01
DDN-0432-0090	Door System Car Monitoring Network Interface Control Document, Rev. 0 – Draft

Echelon Corporation Documents

005-0139-01B	FT 3120 / FT 3150 Smart Transceiver Data Book; 2004
078-0120-01C	LonMark™ Application Layer Interoperability Guidelines, Ver. 3.0
005-0027-01	SNVT Master List and Programmer's Guide, Rev. K
005-0025-01	File Transfer, LONWORKS™ Engineering Bulletin, Rev. D, November 1996
29300	Neuron C Programmer's Guide, Rev. 7
29350	Neuron C Reference Guide, Rev. 5
---	LonMark SNVT Master List, Version 11, Rev. 2, May 2002
---	LonMark Functional Profile: Node Object – SFPTnodeObject, Version 2.0, May 2004

3 SOFTWARE DESIGN

3.1 Hardware Block Diagram

The Neuron Gateway Application is responsible for the bi-directional communication between the DCU and the Central Diagnostic Terminal (CDT) via the Car Network (CN). The Neuron Gateway Application requires a dedicated Neuron chip, FT3150.
Figure 3-1 is a Hardware Block Diagram of the Neuron Gateway Application.

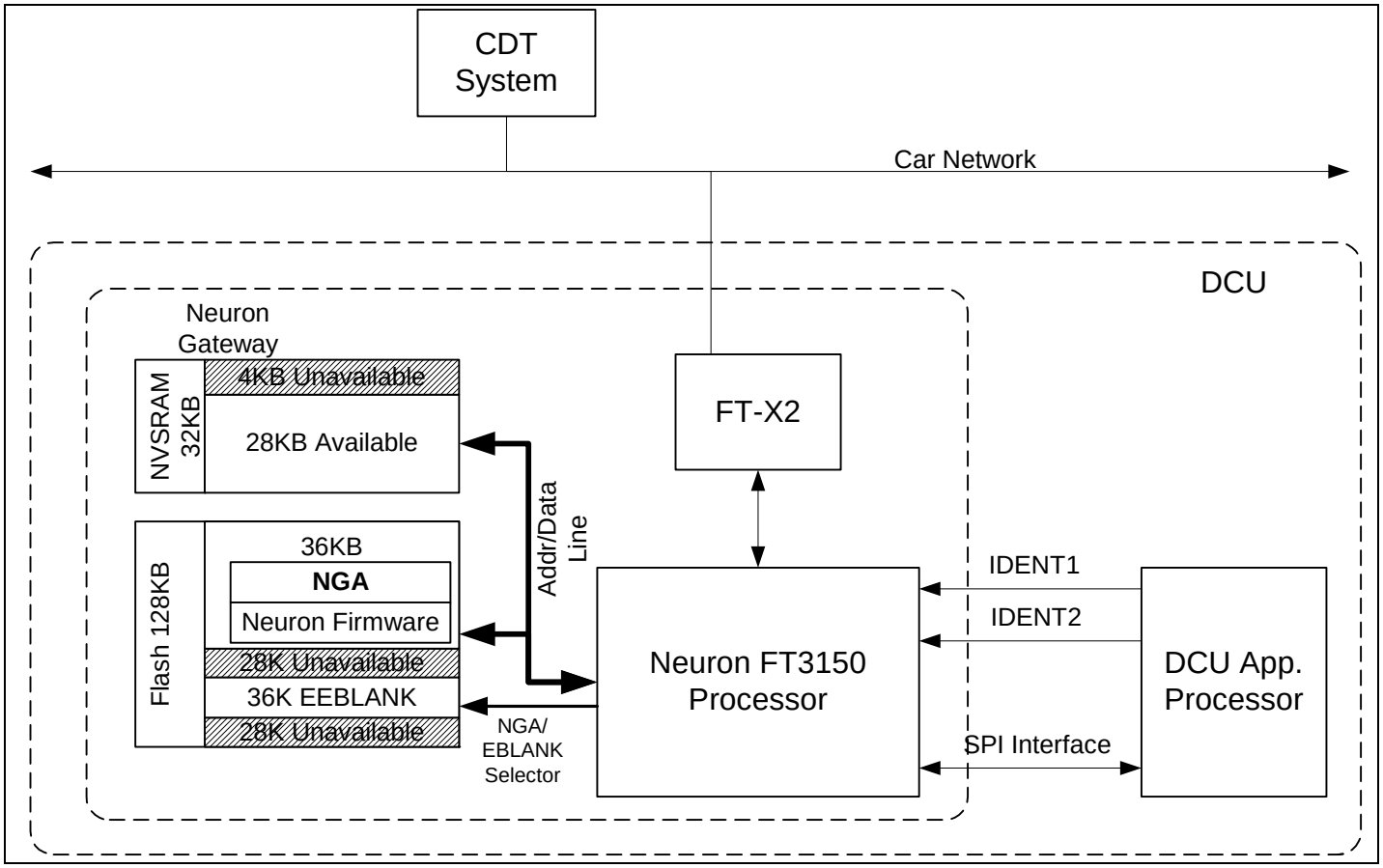

Figure 3-1: Hardware Block Diagram

3.2 Decomposition Description

3.2.1 Entities Decomposition

Figure 3-2 describes the hierarchical decomposition of the Neuron Gateway Application (NGA). NGA consists of the following modules and sub-modules:

- Neuron Firmware
- Initialization Module
- Self Test Module
- Main Process Module
    - SPI Module
    - Status Process Module
- Time Update Module
- FTP Module
- Version Module

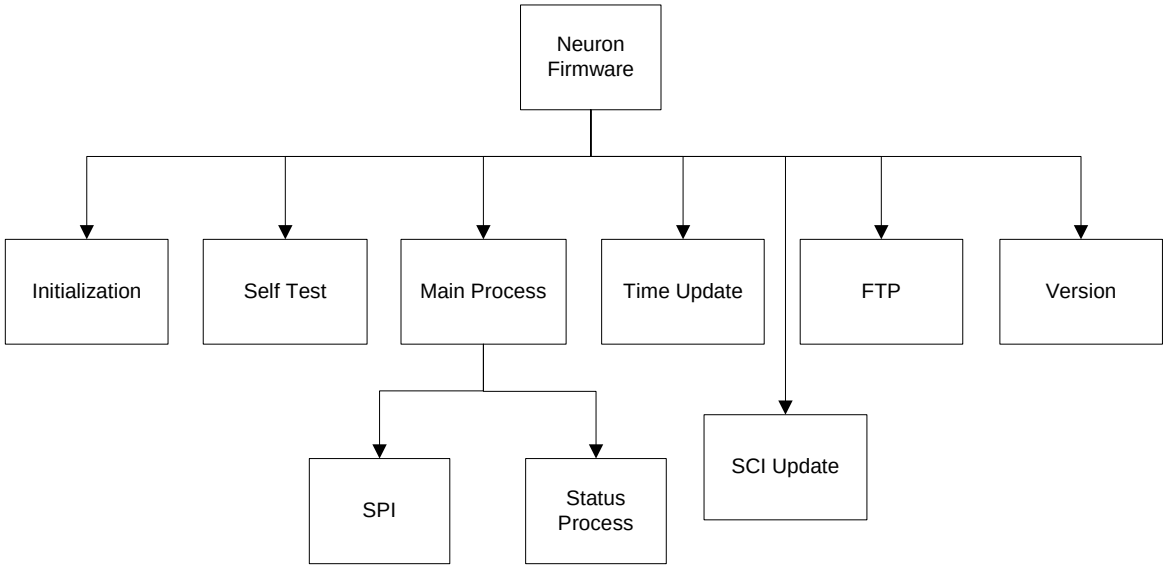

Figure 3-2: Hierarchical Diagram of NGA

3.2.1.1 Neuron Firmware

The Echelon FT3150 firmware (firmware version required by Echelon) is used as the Neuron operating system, which performs task scheduling, system diagnostic, and Neuron CPU hardware I/O.

3.2.1.2 Initialization Module

The Initialization Module performs the following tasks:

- Initialize global variables
- Load identification information, determine DCU location, and initialize the Neuron Domain Table and Address Table
- Initialize the Neuron Configuration Table
- Initialize the Neuron Network Variable Configuration Table

3.2.1.3 Self Test Module

Self Test Module performs the integrity check of the NGA.

3.2.1.4 Main Process Module

The Main Process Module handles the calling of the Neuron main processing loop whenever a defined 50ms timer expires. It schedules the data exchange between Neuron Gateway and DCU Application via SPI as well as monitors the DCU behavior, fault storage, and status processing functions.

3.2.1.5 SPI Module

The SPI Module performs the data exchange, which is predefined in the NGA SPI ICD, document 201111586, between the Neuron Gateway Application and DCU Application.

3.2.1.6 Status Process Module

The Status Process Module provides the CDT with the following DCU status information:

- Operating Status
- Event Set/Reset

3.2.1.7 Time Update Module

The Present Time Module maintains the DCU local date/time according the Car Network broadcast time.

3.2.1.8 SCI Update

The SCI Update Module manages the SCI update request/response and installation status.

3.2.1.9 FTP Module

The FTP Module manages the transfer of the SCI Image file by using LonWorks File Transfer Protocol.

3.2.1.10 Version Module

This module provides the DCU SCI versions when requested by CDT.

3.2.2 Data Decomposition

Figure 3-3 shows the data entities and transfer directions for the Neuron Gateway Application. The following list describes the data type used in the Neuron Gateway Application (see section 3.5 for details):

- SPI Link data packet
- DCU Location
- Network Variable & Explicit Message

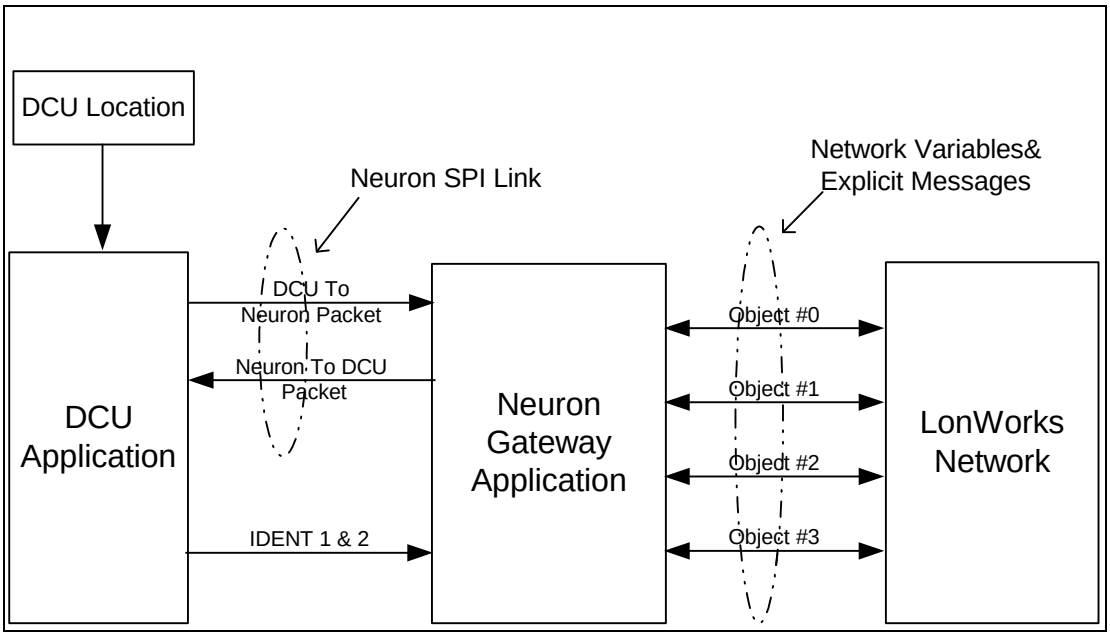

Figure 3-3: Data Decomposition of DCU Application

3.3 Dependency Description

3.3.1 Inter-Entity Dependencies

The Neuron Gateway Application's internal data flow is shown in the following figure, which also describes all the data flow relationships between modules.

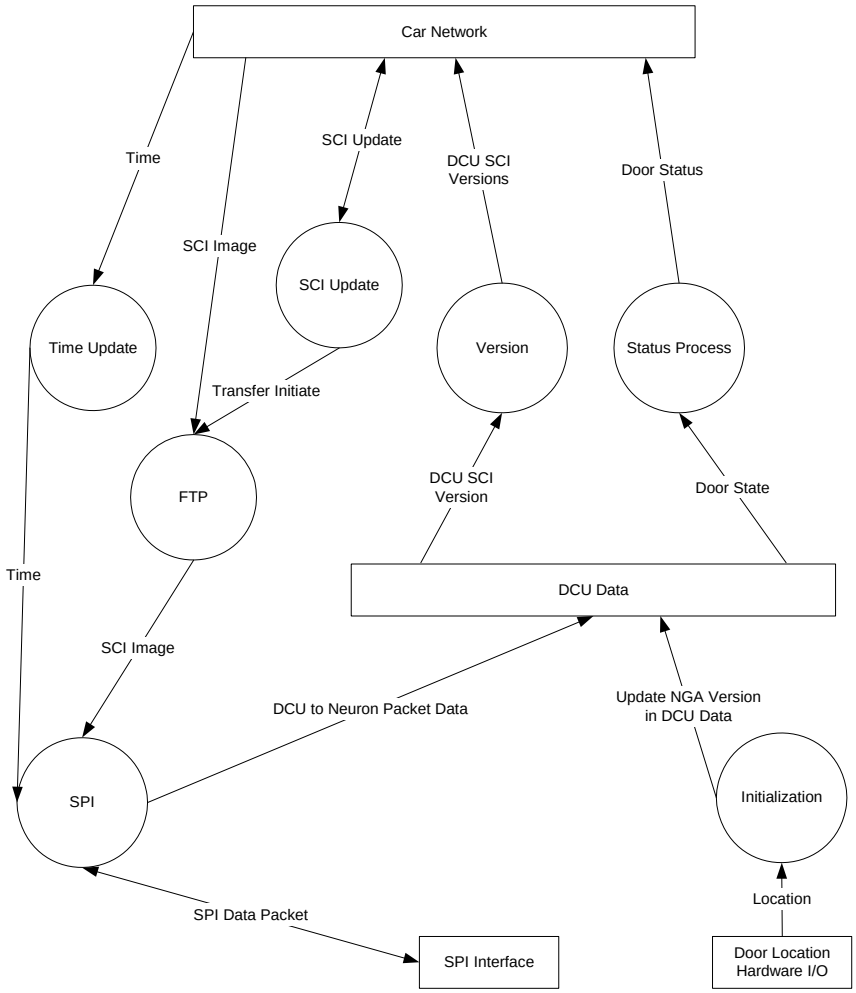

Figure 3-4: NGA Data Flow Diagram

3.3.1.1 Initialization Module

The Initialization module receives the DCU Location information from the Door Location Hardware I/O, and it updates the NGA version in DCU Data.

3.3.1.2 Door Location Hardware I/O

The Door Location Hardware I/O sends the DCU Location to the Initialization module.

3.3.1.3 SPI Module

The SPI module is responsible to send and receive the SPI Data Packet from the SPI Interface. It receives MDS Time from the Time Update module, and then sends these pieces of information over the SPI Interface to the DCU Application. It saves the DCU to Neuron Packet Data after it performs the data exchange with the DCU Application. Finally, it is also responsible for monitoring DCU faults status.

3.3.1.4 DCU Data

It holds the DCU to Neuron Packet Data (DCU Location information and NGA version information). It sends the DCU SCI Version information to the Version Module, and the DCU Signals values to the Status Module.

3.3.1.5 SPI Interface

This is the interface used by the SPI Module to send & receive the SPI Data Packets to & from the DCU Application.

3.3.1.6 Status Process Module

It gets the Door State from the DCU Data and sends the Door Status over the Car Network.

3.3.1.7 Time Update Module

It receives the Present Time packet over the Car Network and sends this to the SPI Module. It also sends the Time information from this packet to the Fault Storage Module.

3.3.1.8 Version Module

It receives the DCU Application Versions from the DCU Data and sends this, along with the Neuron Gateway Application's own version, over the Car Network.

3.3.1.9 SCI Update Module

This module receives the SCI update request and responds to the SCI update status. It also initiates the SCI image transfer to FTP Module.

3.3.1.10 FTP Module

This module receives the SCI image file from the Car Network and sends it to the SPI Module.

3.3.1.11 Car Network

It receives the Door Status from the Status Process, and the DCU SCI Versions from the Version Module. It sends the Time stamp to the Time Update Module, and it also sends the SCI Image to the SCI Update Module.

3.3.2 Inter-Process Dependencies

The Neuron Gateway Application internal control flow is shown in Figure 3-5, which also describes the inter-active relationship between modules.

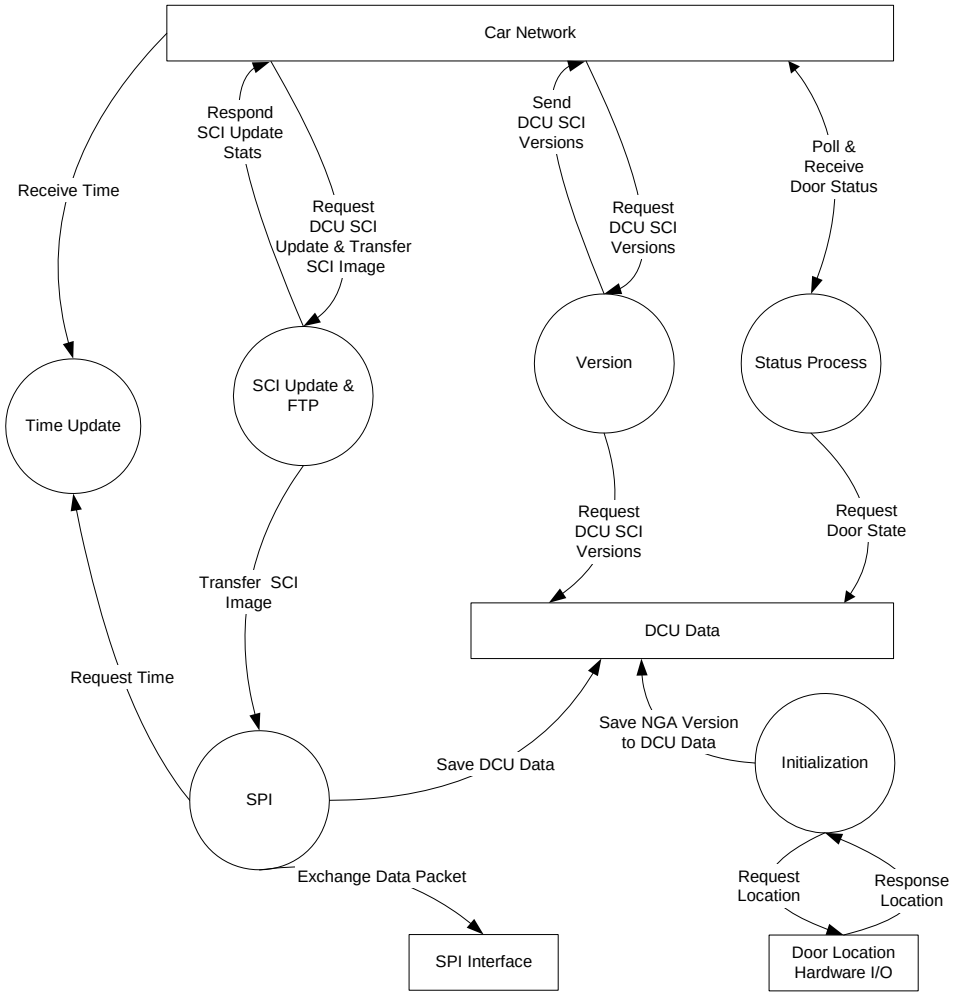

Figure 3-5: Neuron Gateway Control Flow Diagram

3.3.2.1 Initialization

It sends the Request Location command to Hardware I/O to get the DCU location. It also sends the request to Save DCU Data (Location Data and NGA Version/CRC Data) to DCU Data.

3.3.2.2 Door Location Hardware I/O

It receives the Request Location command from the Initialization Module and sends the Location ID.

3.3.2.3 SPI

The SPI Module sends the Exchange Data Packet command to SPI Interface, the Save DCU Data command to the DCU Data, and Request Time command to Time Update Module.

3.3.2.4 DCU Data

It receives the Save DCU Data command from the SPI Module, the Request DCU SCI Version command from the Version Module, Request from the Door State, and the Save NGA Version from the Initialization.

3.3.2.5 SPI Interface

It receives the Exchange Data Packet command from the SPI Module.

3.3.2.6 Status Process

It sends the Request Door State command to the DCU Data, receives the Poll Door Status command from the Car Network, and sends the Send Door Status command to the Car Network.

3.3.2.7 Time Update

The Time Update Module receives the Receive Time command from the Car Network. It also receives the Request Time command from the SPI Module.

3.3.2.8 SCI Update and FTP

The SCI Update Module responds to the Car Network upon receiving the SCI Update request. The FTP Module receives the SCI Image file from the Car Network. It sends SCI Image file to SPI.

3.3.2.9 Version

The Version Module receives the Request DCU Version command from the Car Network. It sends the Request DCU Version command to the DCU Data, and the Send DCU SCI Versions command to the Car Network.

3.3.2.10 Car Network

The Car Network sends the SCI Image to the FTP Module, the DCU Version command from the Version Module. It also sends the Receive Time command to the Present Time Module, the Poll Door Status command to the Status Process, the Request DCU Versions command to the Version Module.

3.3.3 Data Dependencies

See Sections 3.3.1 and 3.3.2.

3.4 Interface Description

3.4.1 Internal Interfaces

3.4.1.1 Main Process Module

Function Name	when (timer_expires(MainLoopTimer))
Description	This function is scheduled to be called once every 100 ms (MainLoopTimer). It performs the SPI data exchange, monitoring of all DCU data, and supervision of SLDT, FTP, & Status transmission to CDT processes.
Input	Location Data and Network Health Data
Output	None
Called from	Neuron Firmware Scheduler

Function Name	when (reset)
Description	This function initializes the Neuron computer system by calling initInitialization().
Input	None
Output	None
Called from	Neuron Firmware Scheduler

3.4.1.2 Initialization Module

Function Name	initInitialization()
Description	This function initializes the Neuron computer system.
Input	Location Data
Output	Location Data and NGA MDS Log flags
Called from	when (reset)

Function Name	initGlobalSignalInit( )
Description	This function initializes various global variables.
Input	NGA MDS Log flags
Output	SPI Health Data, SCI Versions Structure, SCI CRC Data and NGA MDS Log flags.
Called from	initInitialization()

Function Name	initSetPinDirAndReadBack( )
Description	This function determines which set of pins are connected to each other; it also performs diagnostics on the Door Location Hardware I/O circuit.
Input	Door Location Hardware I/O
Output	Door Location Hardware I/O
Called from	initLoadLocation( )

Function Name	initLoadLocation( )
Description	This function determines the location/address of the DCU based on the Hardware I/O.
Input	Door Location Hardware I/O
Output	Location Data
Called from	initInitialization()

Function Name	initSetDomainTable( )
Description	As part of the Node's Self Installation, this function sets up Neuron's Domain Table based on the current DCU location.
Input	Location Data
Output	Neuron's Domain Table and NGA MDS Log flags
Called from	initNeuronNodeConfig( )

Function Name	initSetAddressTable( )
Description	As part of the Node's Self Installation, this function sets up Neuron's Address Table (the addresses with which this Node will communicate) based on the current DCU location.
Input	Location Data
Output	Neuron's Address Table
Called from	initNeuronNodeConfig( )

Function Name	initSetConfigurationStructure( )
Description	As part of the Node's Self Installation, this function sets up Neuron's Configuration Table (Location String, transceiver and other hardware parameters) based on the current DCU location.
Input	Location Data
Output	Neuron's Configuration Table
Called from	initNeuronNodeConfig( )

Function Name	initSetNetworkVariableTables( )
Description	As part of the Node's Self Installation, this function sets up Neuron's Network Variable Table (Network Variable Selector codes, Binding and other Service Type information) based on the current DCU location.
Input	Location Data
Output	Neuron's Network Variable Table
Called from	initNeuronNodeConfig( )

Function Name	initNeuronNodeConfig( )
Description	This function configures and installs the DCU node (Self Installation) by sequentially calling other functions in this Module.
Input	None
Output	None
Called from	initInitialization()

3.4.1.3 Self Test Module

Function Name	selftestNGA16BitCRC( )
Description	This function calculates the 16 bit CRC on the NGA Code in the memory and returns the value to the calling function. This function is only a Power On Self Test feature.
Input	FLASH memory contents
Output	16 bit CRC
Called from	initGlobalSignalInit()

Function Name	selftest16BitsToASCIIHex( )
Description	This function converts a 16 bit unsigned number (a long) to a four character ASCII String representation of the number in hex; it is used for sending back the Self Test Results in the SRS defined format.
Input	16 bit unsigned number
Output	4 character ASCII String
Called from	when (msg_arrives(SELF_TEST_REQUEST))

3.4.1.4 Status Process Module

Function Name	statusDCUToCNC( )
Description	This function sends DCU status information to the CDT.
Input	DCU Signals, SPI Health Data and Location Data
Output	nvoStatDCU
Called from	when (timer_expires(MainLoopTimer))

3.4.1.5 Time Update Module

Function Name	when (msg_arrives(PRESENT_TIME_MSG))
Description	When the Present Time Message (Message Code 20) is received from the Car Network, this function updates the time clock and transmission mode. It also updates Network Health Data and, if needed, updates SLDT status flag.
Input	Present Time
Output	Network Health Data, NGA Present Time and SLDT status flag
Called from	Neuron Firmware Scheduler

Function Name	presenttimeValidateTime()
Description	When the Present Time Message (Message Code 20) is received from the Car Network, this function is used to perform range checking on the values received.
Input	Present Time
Output	Present Time
Called from	when (msg_arrives(PRESENT_TIME_MSG))

Function Name	presenttimeNetworkHealthMonitor()
Description	This function performs periodic checks of the status of network communications.
Input	Present Time and Neuron Firmware's Error Log
Output	Network Health Data
Called from	when (timer_expires(MainLoopTimer))

3.4.1.6 FTP Module

Echelon publication #005-0025-01, Rev. D File Transfer, LONWORKS™ Engineering Bulletin, November 1996, provides the necessary background for most of the functions listed in this subsection.

Function Name	when(nv_update_occurs(nviFileReqD))
Description	This function is either initiates/terminates the FTP process, or it simply returns the status of the File. See Echelon 005-0025-01, Rev. D File Transfer, LONWORKS™ Engineering Bulletin, November 1996
Input	nviFileReqD, File, Present Time, Network Health Data,
Output	FTP Status Information
Called from	Neuron Firmware Scheduler

Function Name	when (resp_arrives(ftp_data_tag))
Description	This function responds to a request from the CDT following a Request/Response FTP data packet sent to the CDT. Depending on the status of the process, it decides on the next step.
Input	FTP Data Packet Response, Network Health Data, Present Time, FTP Status information
Output	FTP Status Information
Called from	Neuron Firmware Scheduler

Function Name	ftpSendNVFileStatD()
Description	This function sends the Status Network Variable nvoFileStatD to the CDT to inform about the status of the File Transfer function.
Input	File and FTP Status information
Output	nvoFileStatD
Called from	ftpStopProcess(), when(nv_update_occurs(nviFileReqD)) and when (resp_arrives (ftp_data_tag))

Function Name	ftpCopyCurrentWindowData()
Description	This function copies the current window's data (the window that is next in line for transmission) from the MDS Log file to a buffer that holds data for up to 6 FTP Data packets.
Input	File and FTP Status information
Output	FTP Status information and data for next window to be transmitted.
Called from	ftpStartDataTransfer() and when (resp_arrives (ftp_data_tag))

Function Name	ftpSendOnePacket()
Description	Send new 6 packets or re-send the missing packets. See Echelon 005-0025-01, Rev. D File Transfer, LONWORKS™ Engineering Bulletin, November 1996
Input	FTP Status information and data for next window to be transmitted.
Output	FTP Data Packet
Called from	ftpStartDataTransfer(), when (resp_arrives (ftp_data_tag)) and when (msg_succeeds (ftp_data_tag))

Function Name	ftpStopProcess()
Description	This function formally closes out the transfer process; the closure may be a result of any of the several reasons (depending on the where this function is being called from)
Input	None
Output	FTP Status information
Called from	when(nv_update_occurs(nviFileReqD)), when (resp_arrives (ftp_data_tag)), when (msg_succeeds (ftp_data_tag)), when (msg_fails (ftp_data_tag)) and when (timer_expires (FTPProblemsTimer))

Function Name	ftpStartDataTransfer()
Description	This function is responsible for starting the transfer sequence; it initializes the FTP Status information
Input	None
Output	FTP Status information
Called from	when(nv_update_occurs(nviFileReqD))

Function Name	when (msg_succeeds(ftp_data_tag))
Description	This function facilitates the continuation of the FTP process after successfully sending of the outgoing FTP data packet that does not require a formal response from the CDT
Input	Neuron Firmware Events, FTP Status Information, Network Health Data and Present Time
Output	FTP Status information
Called from	Neuron Firmware Scheduler

Function Name	when (msg_fails(ftp_data_tag))
Description	This function stops the transfer process (by calling other functions) if there is a failure signaled by the Neuron Firmware
Input	FTP File Status
Output	FTP File Status
Called from	Neuron Firmware Scheduler

Function Name	when (timer_expires(FTPProblemsTimer))
Description	This function stops the transfer process (by calling other functions) if there is a failure detected internally by the NGA
Input	FTP File Status
Output	FTP File Status
Called from	Neuron Firmware Scheduler

3.4.1.7 Version Module

Function Name	when (msg_arrives(SW_VERSION_REQUEST))
Description	When the Software Version Request Message (Message Code 21) is received from the Car Network, this function sends the DCU SCI versions to the Car network.
Input	Software Version Request and Present Time
Output	Software Version Response (Message Code 22)
Called from	Neuron Firmware Scheduler

Function Name	versionConvertByteToDecimalString()
Description	This function converts a byte to a decimal String. The String could be up to 3 characters long (depending on the # of decimal digits).
Input	The byte to be converted to decimal string
Output	Converted string
Called from	initGlobalSignalInit()

3.4.1.8 SPI Module

Function Name	spiDataExchange()
Description	This function performs the Data Exchange over the SPI interface when it is called once every 100 ms. It performs these functions by sequentially calling other functions (from preparing the outgoing packet to processing the incoming packet).
Input	SPI Communication Buffer
Output	SPI Communication Buffer
Called from	when (timer_expires(MainLoopTimer))

Function Name	spiNeuronBufferPrepare()
Description	The function prepares and formats the outgoing packets to the DCU Application.
Input	Present Time, Network Health Data, Location Data, MDS Log File, SCI Versions Structure, Partial Close & Local Recycle commands and SPI Health Data.
Output	SPI Communication Buffer
Called from	spiDataExchange()

Function Name	spiIncomingDataCheck()
Description	The function validates the incoming packets for Checksum and proper synchronization, and then updates various structures by calling other functions in this module.
Input	SPI Communication Buffer
Output	SPI Health Data and MDS Log File
Called from	spiDataExchange()

Function Name	spiUpdateDCUStructs()
Description	The function updates the DCU Signals structure with the data received over the SPI interface.
Input	SPI Communication Buffer
Output	DCU Signals
Called from	spiIncomingDataCheck()

Function Name	spiUpdateSCIVersionsStructs()
Description	The function updates the SCI Versions Structure & SCI CRC Data with the data received over the SPI interface every time a Level 2 frame is complete and the contents have been range checked.
Input	SPI Communication Buffer
Output	SCI Versions Structure and SCI CRC Data
Called from	spiIncomingDataCheck()

Function Name	spiChecksum()
Description	This function returns the simple unsigned Checksum + 1 for data pointed by a pointer and for the number of bytes indicated.
Input	Data pointer and Number of bytes for which checksum + 1 is desired
Output	Checksum + 1 for the input
Called from	spiNeuronBufferPrepare() and spiIncomingDataCheck()

3.4.2 External Interfaces

3.4.2.1 Door Location Hardware I/O

There are 2 inputs of the Neuron Input mapping to the 2 pins of the Car network connector J3, which are used to determine the location of the unit in the car system:

- IDENT1
- IDENT2

The DCU location is shown in Table below.

Table 3-1: DCU Location

Connector Layout (IDENT1:IDENT2)	DCU Location	Description
0:0	DCU R1	Right side, front end
0:1	DCU L2	Right side, rear end
1:0	DCU R2	Left side, front end
1:1	DCU L1	Left side, rear end

3.4.2.2 SPI Interface

Details of this interface are provided in Neuron SPI ICD document 201111586.

3.4.2.3 Car Network

The following table lists all network variables and explicit messages used in the DCU Application (For more details, see NGA SRS document 201111256):

Table 3-2: Network Variable List

Entry	Variable	Direction	Priority	Service
1	nvi00ObjRequest	0 (IN)	0	ACKD
2	nvo00ObjStatus	1 (OUT)	0	ACKD
3	nvi00TimeStamp	0 (IN)	0	UNACKD_RPT
4	nvi00FileReq	0 (IN)	0	ACKD
5	nvo00FileStat	1 (OUT)	0	ACKD
6	nvi01Subscribe	0 (IN)	0	ACKD
7	nvi01TransfState	0 (IN)	0	UNACKD
8	nvo01StatusBits	1 (OUT)	0	ACKD
9	nvo01Parameter	1 (OUT)	0	UNACKD
10	nvi02SCIUpdReq	0 (IN)	0	ACKD
11	nvo02SCIUpdResp	1 (OUT)	0	ACKD
12	nvi03SCIVersReq	0 (IN)	0	ACKD
13	nvo03SCIVersion	1 (OUT)	0	ACKD

3.4.3 Data Dictionary

See Sections 3.4.1, 3.4.2, and 3.5.2.

3.5 Detailed Design

3.5.1 Entities Detailed Design

3.5.1.1 SD3.1 Initialization Module

Each time the system is rebooted, the Neuron Gateway Application will perform an initialization. During this initialization, the NGA will perform the Neuron Gateway system self-test, reinstall the network configuration, and initialize the global variables.

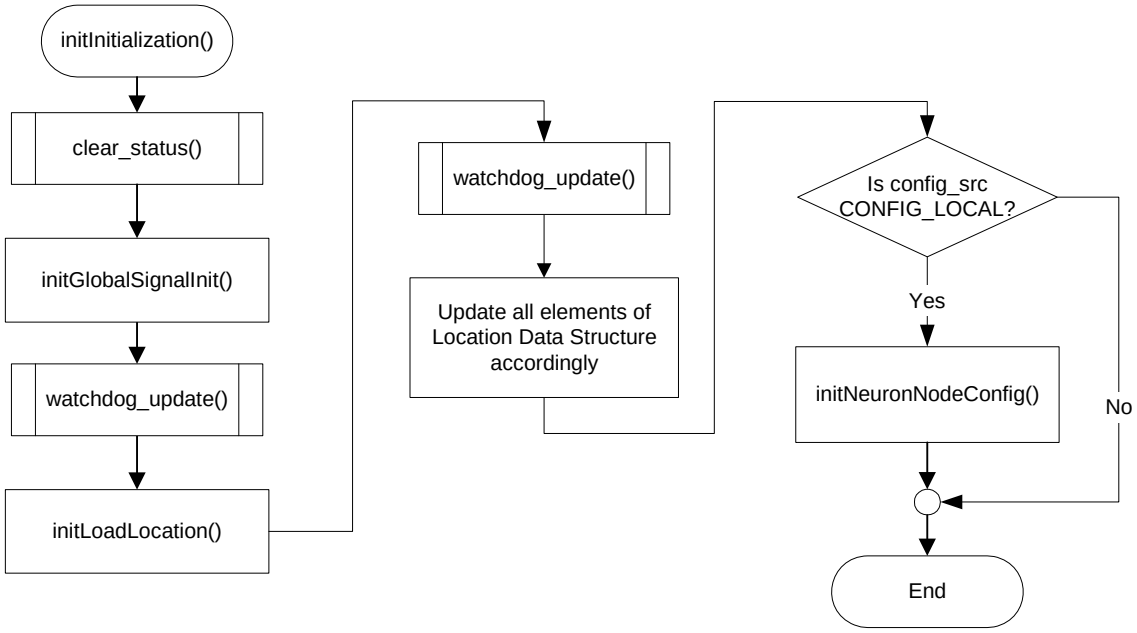

Figure 3-6: initInitialization Routine Flowchart

The watchdog_update function is a Neuron C built-in function used to update watchdog timer.

The function initGlobalSignalInit() initializes global variables sequentially as follows:

- Set SPI Health Data (bBadSPI) to the default of 1 for DCU Application.
- Set Network Health Data (bNLC) to the default of 1.
- Set non-changing fields in SCI Version Structure (usEq_Name, usSupplier, usSCI_Info, usSCI_Name and NGA's usVersion).
- Set non-changing fields in Self Test Response Packet (usSCI_Info and the 4 blank spaces in usCRC fields).

Additionally, initGlobalSignalInit() performs a 16 bit CRC check on the FLASH memory to check the health of its code space by calling the function selftestNGA16BitCRC(). The selftestNGA16BitCRC() function uses the Neuron C built-in function crc16() to perform its calculation. If the CRC check fails, the NGA will cause the Neuron processor to go Offline.

The initLoadLocation() function returns the DCU Location information. The hardware DCU identification bits help NGA to determine the DCU position on the train. See Table 3-1 for details about the locations and the respective identification bits.

Once the DCU Location information is determined, the function initInitialization() shall call the function initNeuronNodeConfig() to configure and self-install the Neuron Node according to the following:

- Configuration Structure

- Domain
- Address
- Network Variable tables

Note: The setting of Program ID to “DCU” is not done thru any of the functions; it is set in the compiler.

Table 3-3: Neuron Configuration Structure

Offset	7	6	5	4	3	2	1	LSB 0
0	channel_id = 0							
1								
2	The Location ID String is based on the DCU Location							
3								
4								
5								
6								
7								
8	comm_clock = 4					input_clock = 5		
9	comm_type = 1			comm_pin_dir = 14				
A	preamble_length = 8							
B	packet_cycle = 5							
C	bata2_control = 12							
D	xmit_interpacket = 14							
E	recv_interpacket = 15							
F	node priority = 0							
10	channel priority = 4							
11	CD = 0	bitsync = 0		filter = 0		hysteresis = 0		
12	cd_to_end_packet = 41						cd_t = 0	cdP = 0
13								
14								
15								
16								
17								
18	non_group_timer = 5				auth = 0	preamption_timeout = 5		

The Configuration Structure is set by initNeuronNodeConfig() function calling initSetCofigurationStructure(). The initSetCofigurationStructure() in turn shall use the built-in Neuron C function update_config_data().

Table 3-4: Neuron Domain Table

Offset	7	6	5	4	3	2	1	LSB 0
0	Domain ID = 0							
1	Domain ID = 0							
2	Domain ID = 0							
3	Domain ID = 0							
4	Domain ID = 0							
5	Domain ID = 0							
6	Subnet ID = 1							
7		Node ID based on the DCU Location: Location ID — DCU Location 0x11 — DCU R1 0x12 — DCU R2 0x13 — DCU L1 0x14 — DCU L2						
8	Domain ID Length = 0							
9	Authorization Key = 255							
A	Authorization Key = 255							
B	Authorization Key = 255							
C	Authorization Key = 255							
D	Authorization Key = 255							
E	Authorization Key = 255							

The Domain Table is set by initNeuronNodeConfig() function calling initSetDomainTable ().
The initSetDomainTable () in turn shall use the built-in Neuron C function update_domain ().

Table 3-5: Neuron Address Table

Entry	Type	Domain	Node	Repeat Timer	Retry	Receive Timer	Transmit Timer	Subnet
0	1	0	1	0	3	0	4	1
1	1	0	1	0	3	0	4	2
2	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	0
6	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0
9	0	0	0	0	0	0	0	0
10	0	0	0	0	0	0	0	0
11	0	0	0	0	0	0	0	0
12	0	0	0	0	0	0	0	0
13	0	0	0	0	0	0	0	0
14	0	0	0	0	0	0	0	0

The Address Table is set by initNeuronNodeConfig() function calling initSetAddressTable (). The initSetAddressTable () in turn shall use the built-in Neuron C function update_address ().

Table 3-6: Network Variables List Table

Entry	Variable	Direction	Service	Selector (Note)*	Address Index
0	nci00NetConfig	0 (IN)	ACKD	0x1?16	0x0F
1	nvi00ObjRequest	0 (IN)	ACKD	0x1?01	0x0F
2	nvo00ObjStatus	1 (OUT)	ACKD	0x1?02	0x00
3	nvi00TimeStamp	0 (IN)	UNACKD_RPT	0x0803	0x0F
4	nvi00FileReq	0 (IN)	ACKD	0x1?04	0x0F
5	nvo00FileStat	1 (OUT)	ACKD	0x1?05	0x0F
6	nvi01Subscribe	0 (IN)	ACKD	0x1?06	0x0F
7	nvi01TransfState	0 (IN)	UNACKD	0x0109	0x0F
8	nvo01StatusBits	1 (OUT)	ACKD	0x1?0A	0x00
9	nvo01Parameter	1 (OUT)	UNACKD	0x1?0C	0x00
10	nvi02SCIUpdReq	0 (IN)	ACKD	0x1?0D	0x0F
11	nvo02SCIUpdResp	1 (OUT)	ACKD	0x1?0E	0x0F
12	nvi03SCIVersReq	0 (IN)	ACKD	0x1?0F	0x0F
13	nvo03SCIVersion	1 (OUT)	ACKD	0x1?10	0x00

*Note: DCU node HI selectors (0x1?) are set to 0x11 through 0x14 for installation locations 1 to 4. Selectors for nvi00TimeStamp and nvi01TransferState are the same for all Door Nodes.

The Network Variables Table is set by initNeuronNodeConfig() function calling initSetNetworkVariableTables (). The initSetNetworkVariableTables () in turn shall use the built-in Neuron C function update_nv().

3.5.1.2 SD3.2 Main Process Module

The when(timer_expires(MainLoopTimer)) function executes every 50 ms according to MainLoopTimer programmed value. Figure 3-7 below provides information about this function.

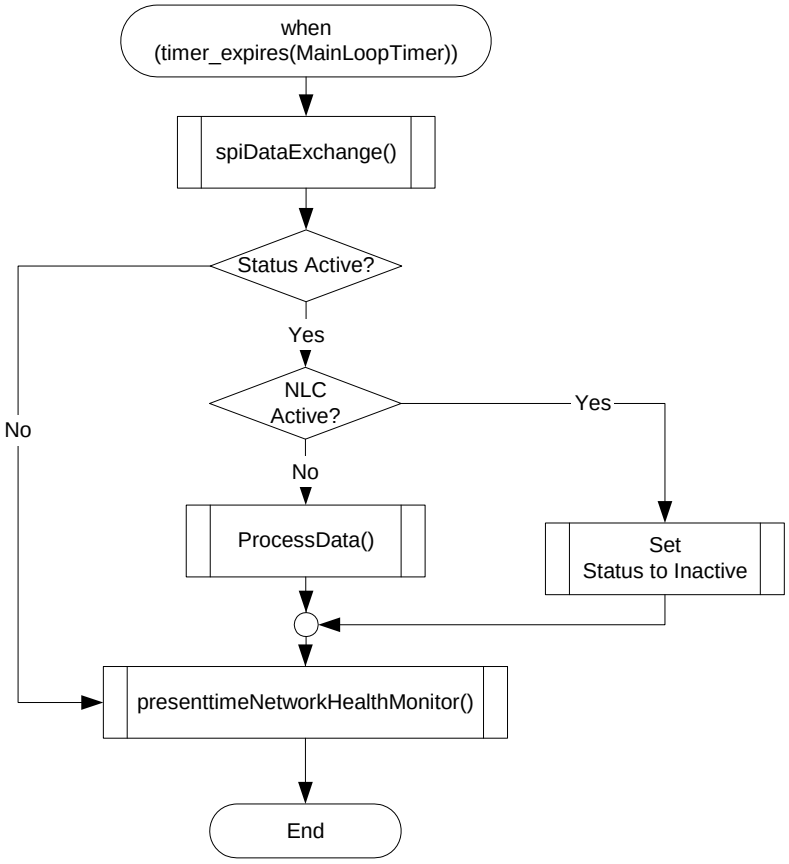

Figure 3-7: when(timer_expires(MainLoopTimer)) Flowchart

The other function in the Main module is when (reset), which is executed at boot up of NGA. This function simply calls the initInitialization() function.

3.5.1.3 SD3.3 Self Test Module

The two functions in this module are the relatively straight forward selftestNGA16BitCRC() and selftest16BitsToASCIIHex() functions. The former just calls the Neuron C's built-in function crc16() to compute the CRC on the FLASH memory, while the latter just parses thru to convert a 16 bit unsigned number to a four character ASCII string representation of the number in hexadecimal format.

3.5.1.4 SD3.4 SPI Module

The SPI Module is responsible for the data exchange between the Neuron Gateway and DCU Application. The data being sent to the DCU Application is prepared in the SPI_Neuron_Buffer, and the data received from DCU Application is saved in the SPI_Buffer. The Checksum is verified; if a Checksum failure occurs, the message is discarded.

3.5.1.4.1 SD3.4.1 Neuron's Built-in I/O Object

The Neuron Firmware's built-in I/O function (along with the appropriate SPI I/O object declaration) is used to handle the low level data exchange over the SPI interface; this built-in SPI function/object suspends Neuron Gateway Application processing during the period the exchange is taking place. For this purpose, the SPI object shall be declared as follows:

```
IO_8 neurowire master select(IO_7) spiDataIO;
```

The baud rate is left out of the above declaration to default to 20 kbps, and the clock edge is left out to default to rising edge clocking (+).

Additionally, pin IO_7 shall be declared as follows as a synchronization output for the DCU Application (this output shall be turned on before start of data exchange, and shall be turned off immediately following completion of the exchange):

```
IO_7 output bit spiSync = 0;  //Initialize to off
```

The built-in function shifts out bits (MSB first) from buffer pointed to by the pointer in the function call, while simultaneously shifting in the incoming data into the same buffer from the other end. The built-in function allows up to 255 bits to be exchanged per function call. With 40 bytes to be sent to each DCU Application, while 40 bytes are received from the DCU Application, the exchange shall be split into two packets of 20 bytes (160 bits) each. With this in mind, the syntax for calling the built-in function shall be:

```
io_out (spiDataExchange, SPI_Neuron_Buffer, 160);
```

where, the buffer pointer has to be incremented by 20 for the 2nd of the two packets to point to the beginning of the 2nd half of the buffer.

3.5.1.4.2 SD3.4.2 SPI Data Exchange

The spiDataExchange() function, which is called every 100 ms by the when(timer_expires (MainLoopTimer)) function, performs the high level data exchange over the SPI interface. It calls the Neuron Firmware's built-in I/O function to perform the low level SPI data exchange. Additionally, it calls the following functions:

- spiNeuronBufferPrepare() to prepare the outgoing buffer.
- spiIncomingDataCheck() to verify the integrity of the data coming back from the DCU Application.

The spiIncomingDataCheck() function, in turn, calls the following functions:

- spiUpdateDCUStructs() to update the various DCU data structures.
- spiUpdateSCIVersionsStructs() to update information regarding SCIs in the DCU.
- spiUpdateMDSLog() to update MDS Log.

The process flowchart for the spiDataExchange() function shall be as shown in Figure 3-8.

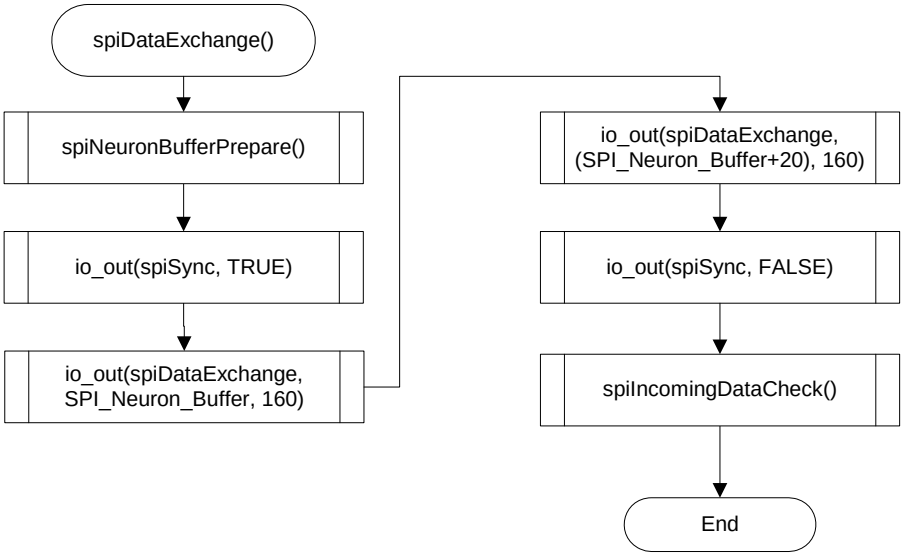

Figure 3-8: spiDataExchange Flowchart

The process flow chart for the spiNeuronBufferPrepare() function shall be as follows:

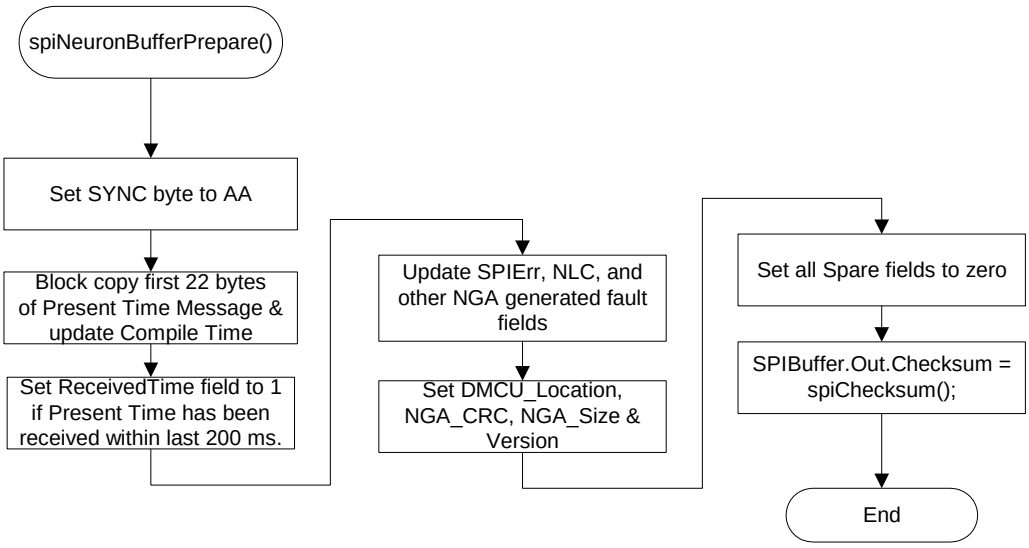

Figure 3-9: spiNeuronBufferPrepare Flowchart

The spiChecksum() function mentioned in the figure above is a function that returns the simple unsigned Checksum + 1 for data pointed to by the argument. This function is used by spiIncomingDataCheck() function also.

The process flowchart for the spiIncomingDataCheck() function shall be as follows:

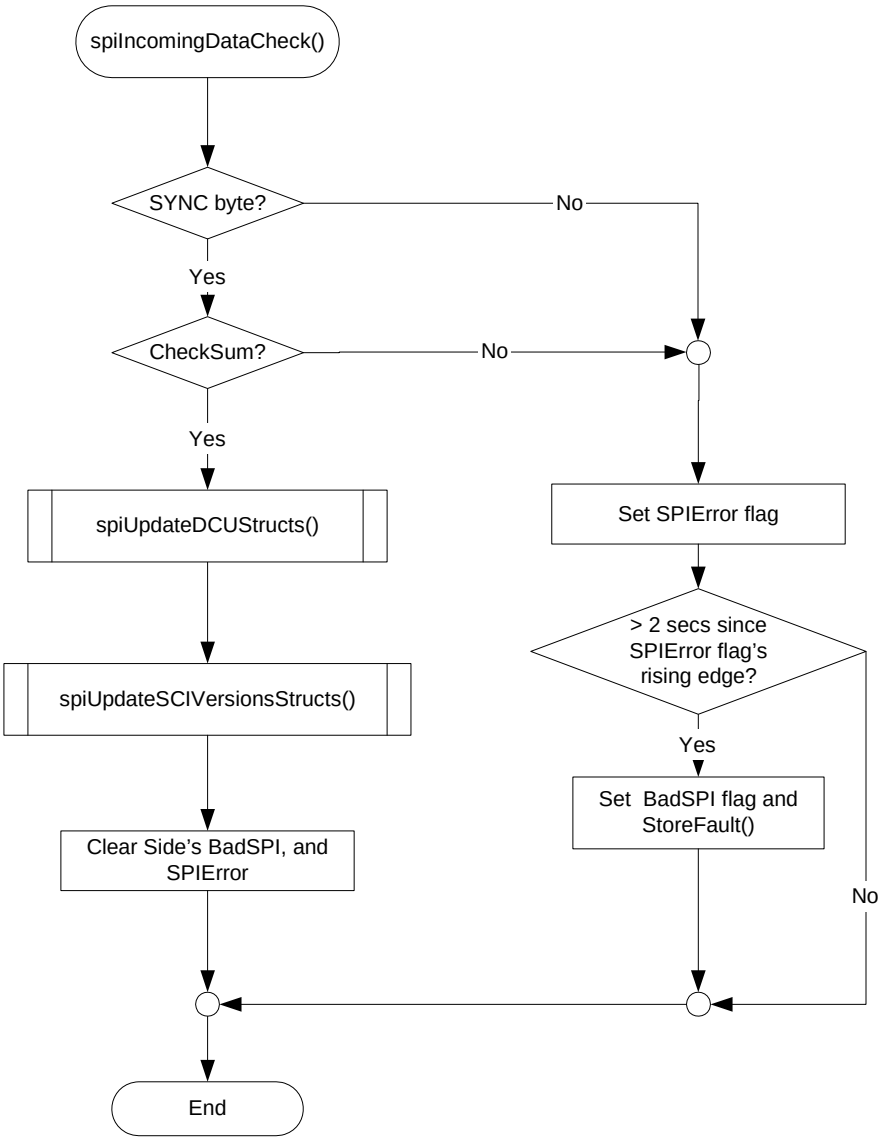

Figure 3-10: spiIncomingDataCheck Flowchart

The spiUpdateDCUStructs() function is responsible for updating the NGA's copy of DCU signals received from the DCU Application. Every time these signals are received, this function updates the corresponding fields in the DCU Signals Structure. Since the layout of DCU Signals Structure is similar to part of the SPI frame that carries this incoming information, a block copy can be used to efficiently update the data. The process flow chart for the spiUpdateDCUStructs() function shall be as follows:

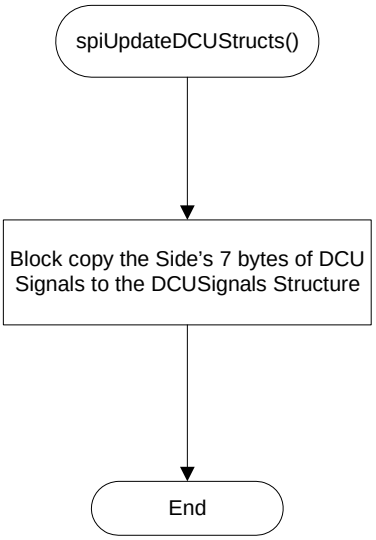

Figure 3-11: spiUpdateDCUStructs Flowchart

The spiUpdateSCIVersionsStructs() function is responsible for monitoring and recording the Level 2 data received from DCU Application. Every time a complete sequence of ExtraDataIDs is received, this function verifies the data and updates the corresponding entries in the SCI Versions Structure. The process flowchart for the spiUpdateSCIVersionsStructs() function shall be as follows:

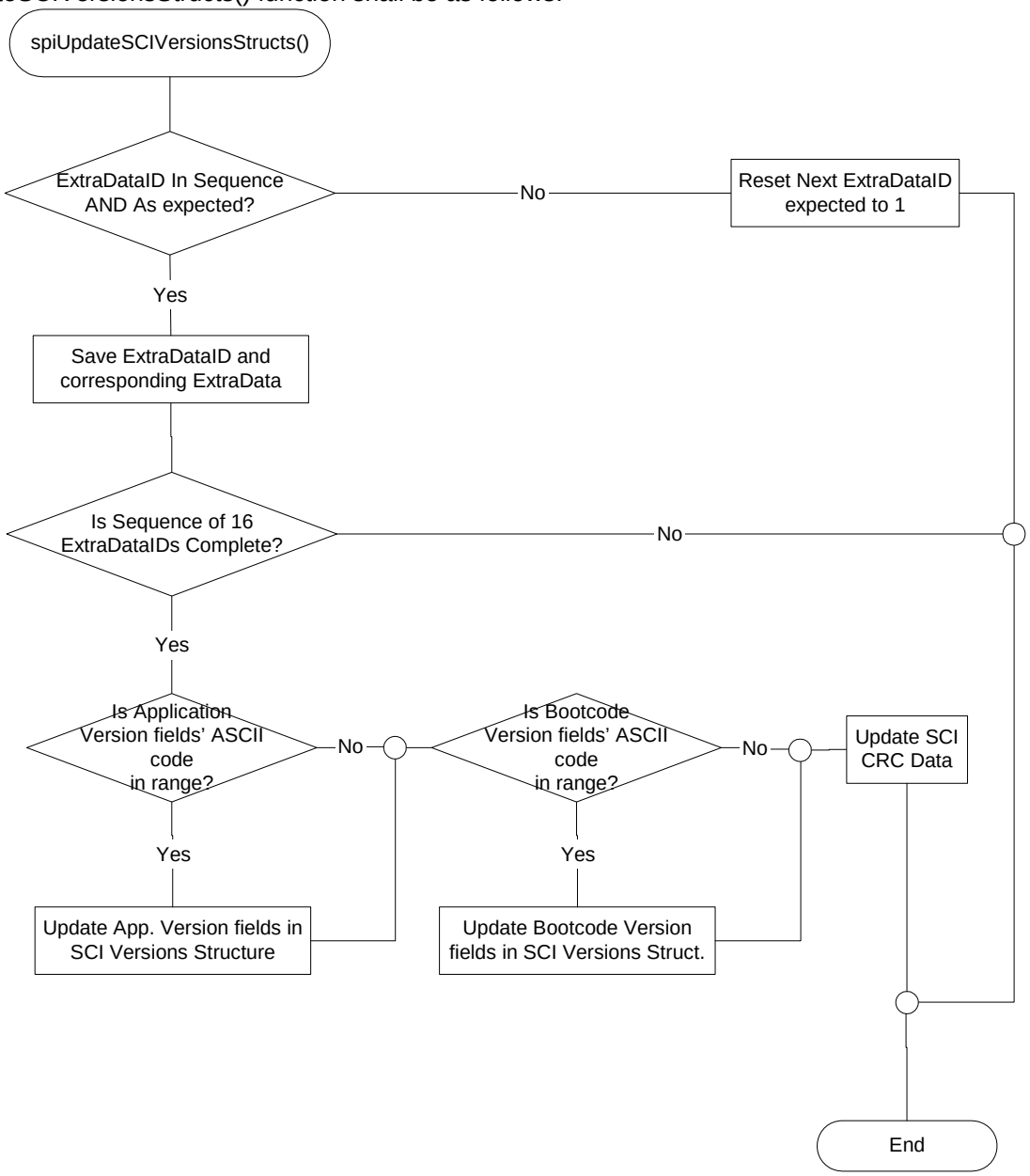

Figure 3-12: spiUpdateSCIVersionsStructs Flowchart

3.5.1.5 SD3.5 SCI Update Module

Figure 3-13 shows the SCI Update process:

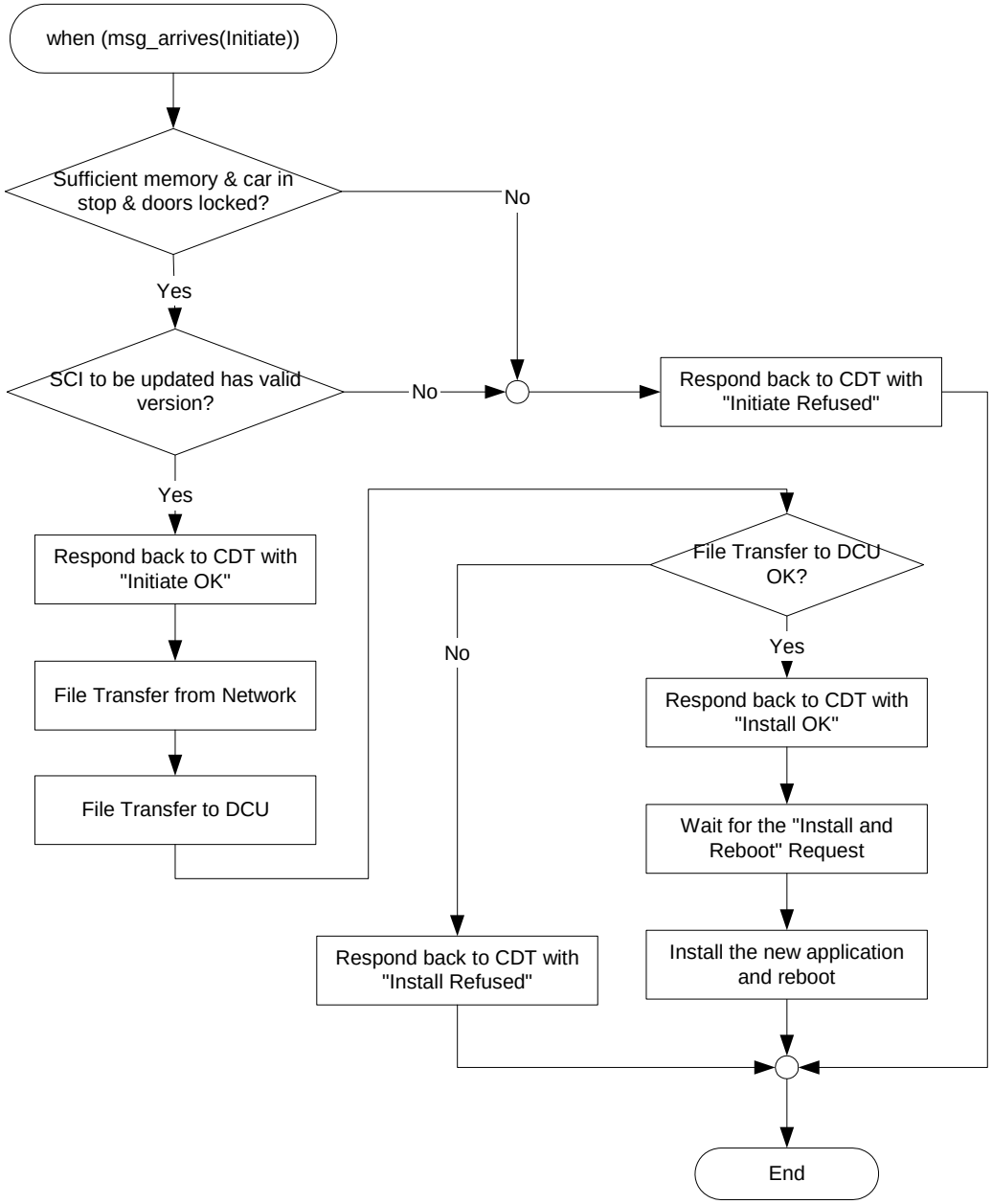

Figure 3-13: Update SCI Flowchart

3.5.1.6 SD3.6 Status Process Module

The Neuron Gateway Application updates the DCU Status variable nvo01StatusBits. The subscribe object request nvi01Subscribe changes the rate of the DCU status transmission. The default rate is 9 seconds, and NGA updates the transmission rate according to the nvi01Subscribe.update_rate value. The nvi01Subscribe.update_rate value must be multiples of 500ms over a range from 0 to 60s. When the nvi01Subscribe.update_rate value is zero, NGA stops the transmission of Status bits.

3.5.1.7 SD3.7 Network Health Module

NGA verifies the Network connection status over 60 seconds. If there is no Network communication, it sets “NLC” to one on the SPI_Neuron_Buffer.

3.5.1.8 SD3.11 Time Update Module

When the Time Update message is received, the Neuron Gateway Application updates the current time. The Time Update contents are also copied to SPI_Neuron_Buffer and sent to DCU Application.

The process flowchart for the main function in this module shall be as follows:

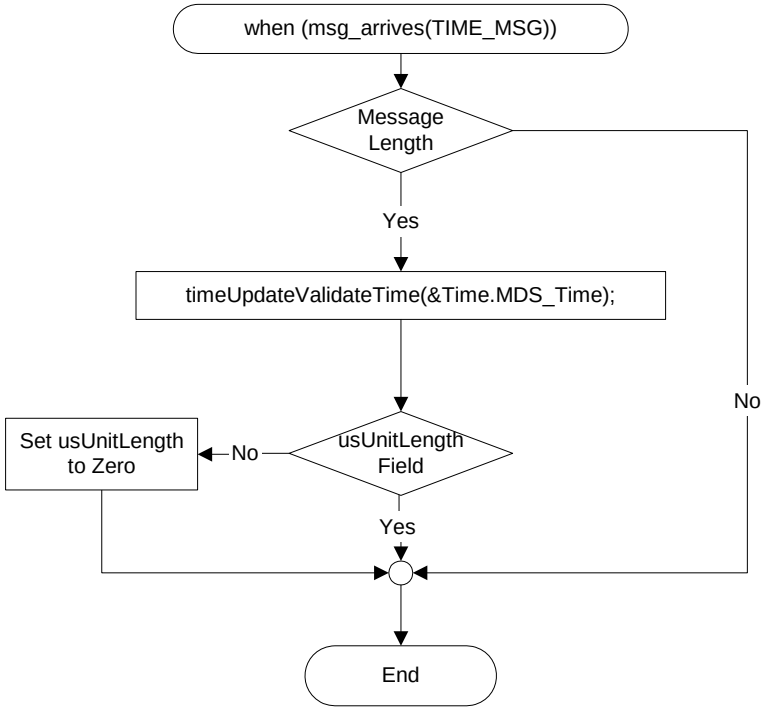

Figure 3-14: Time Update Flowchart

The Time Update Module has a function called timeUpdateValidateTime() that is used to validate the fields received in the Time Update packet from the CDT. This function performs a range change on the fields and if any of the fields are not within the valid ranges, the data and time is set to a default of January 01, 1990, 00 hours, 00 minutes and 00 seconds.

3.5.1.9 SD3.12 FTP Module

Data exchange between nodes in the network is modeled as a file transfer, where a file is a stream of bytes, accessed sequentially either for read or for write. Three logical nodes participate in a file transfer operation: the Initiator, the Sender, and one or more Receiver. The Initiator cooperates with the Sender and the Receiver, and could be the Sender or one of Receivers.
There are two Standard Network Variable Types used to set up a file transfer. SNVT_file_req is used for communication from the Initiator to the Sender and Receiver. SNVT_file_status is used for communication from the Sender and Receiver to the Initiator.

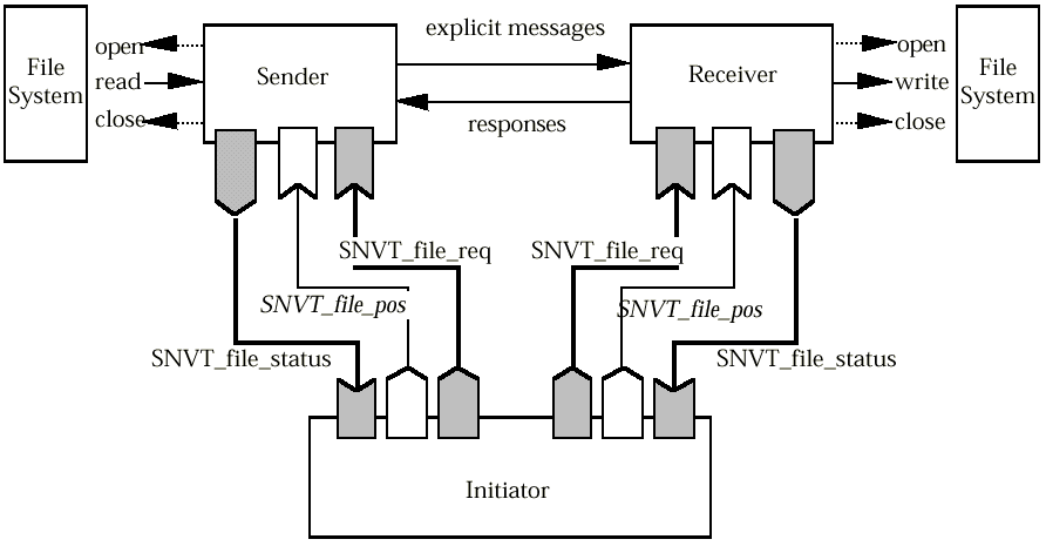

Figure 3-15: File Transfer Architecture

To start the transfer, the Initiator updates the network variable of type SNVT_file_req on the Receivers in order to open the destination files for writing. The result of the file open operation (network variable of type SNVT_file_status) is returned in an update to the Receivers. If all the Receiver files were successfully opened, the Initiator then uses the same procedure to open the source file on the Sender for reading. At the same time, the Initiator passes the network address of the Receivers to the Sender. The Sender uses this address to pass the file data in an explicit message to the Receivers. At the conclusion of the transfer, the Sender informs the Initiator that it has transmitted the whole file, and the Initiator then informs the Sender to close and save the files.
There's 32 bytes data package transferred between the Sender and the Receiver so standard buffer sizes may be used on each node. It applies the windowed transfer protocol mechanism. Most data packages are sent with an unacknowledged service, with a request/response packet sent every six packets. These five unacknowledged packets and the sixth request/response packet constitute a data window. This windowing protocol avoids the overhead of acknowledging every packet, but allows recovery from a lost packet no more than six packets later. The response from the Receivers to the Sender includes an indication of the last packet in the window that was successfully received, incremented by one. The Sender needs to buffer the last data window, in case it receives a request from the Receiver to retransmit one or more packets from that window.

The following is an example with no error shown:

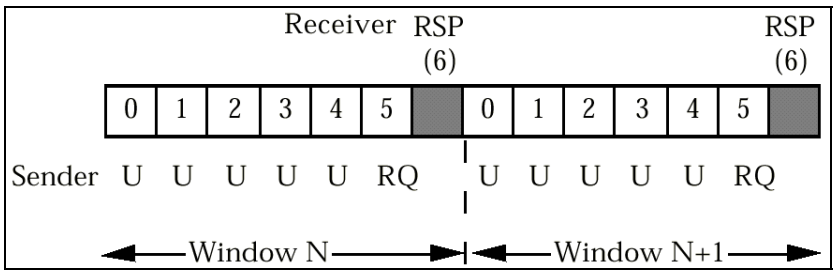

Figure 3-16: File Transfer – Window Successfully Received

The following is an example with error recovery:

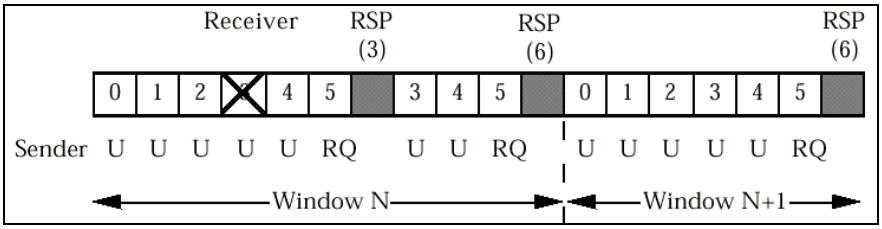

Figure 3-17: File Transfer – Window With Retransmission

3.5.1.10 SD3.14 Version Module

When the Software Version Request message is received, the Neuron Gateway Application sends the Software Version Response message to the Car Network. The message includes the following SCI version information:

- Neuron Gateway Application
- DCU Boot Loader Application
- DCU Application
- CPLD Binary File

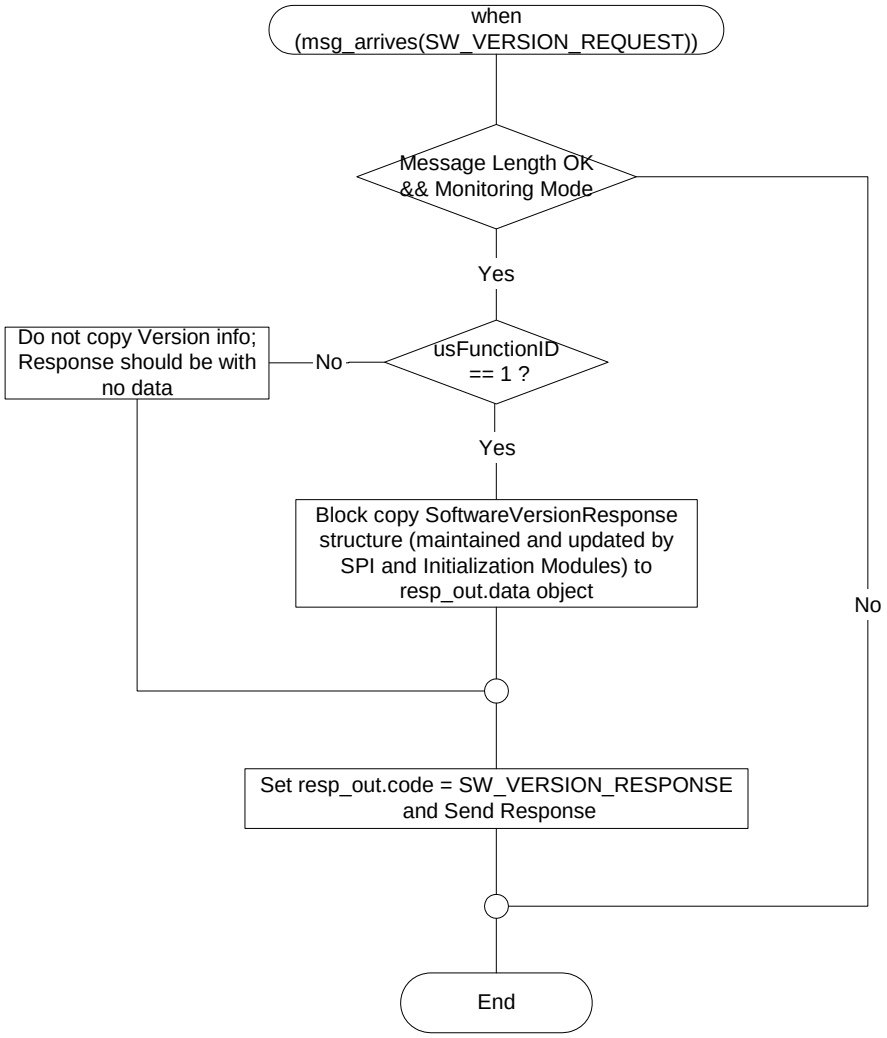

Figure 3-18: when (msg_arrives(SW_VERSION_REQUEST)) Flowchart

This module also has another function called versionConvertByteToDecimalString(); this relatively straightforward function converts a byte (passed to it) and returns the equivalent string that is null terminated.

3.5.2 Data Entities Detailed Design

The following subsections describe the key data entities that are shared by multiple modules of the NGA. The interface is defined in the QT's Door System CMN ICD document DDN-0432-0090.

3.5.2.1 SD3.15 Network Variables

QT's Door System CMN ICD document DDN-0432-0090 contains details of the Network Variables.

Also, see Echelon document 005-0027-01, SNVT Master List and Programmer's Guide, for all SNVT network variables data structures.

3.5.2.2 SD3.16 SPI Communication Buffer

As detailed in the Neuron SPI ICD document 201111586, two SPI communication data structures are to be used for managing the sending and receiving of data over the SPI interface. Since Neuron's SPI interface uses the same physical buffer for the outgoing as well as the incoming data (outgoing data is rotated out from one end, while the incoming data is rotated in from the other end), the 40-byte SPI data communication buffer will be a Union of SPI_Buffer structure followed by SPI_Neuron_Buffer structure.

3.5.2.2.1 SD3.16.1 SPI_Buffer Structure

This 40-byte structure holds the data received from the DCU. The structure is the same as defined in the Neuron SPI ICD document 201111586.

3.5.2.2.2 SD3.16.3 SPI_Neuron_Buffer Structure

This 40-byte structure holds the data that will be sent to DCU. The structure is the same as defined in the Neuron SPI ICD document 201111586.

3.5.2.3 SD3.18 SCI Versions Structure

This structure is used to store all DCU SCI versions, and to help send this information to the CDT: Neuron Gateway Application, Boot Loader, DCU Application, and CPLD Binary File.

```
struct{
      struct{
            unsigned short    usSCI_Info;       // SCI Information
            unsigned short    usEq_Name[10];    // Equipment Name
            unsigned short    usSupplier[12];   // Supplier Name
            unsigned short    usSCI_Name[14];   // SCI Name
            unsigned short    usVersion[6];     // Software Version
      } struct_sver[5];
} SoftwareVersionResponse;
```

3.5.2.4 SD3.19 DCU Signals

This structure is used to store the hardware signals read by the DCU and received from the SPI_Buffer every 100ms. The SLDT sampling and all other software functions read this structure to obtain the current state of the hardware signals.

```
typedef struct
{
    unsigned          ZeroSpeedPWR      :1;
    unsigned          ZeroSpeedRTN      :1;
    unsigned          OpenTLCMD         :1;
    unsigned          CloseTLCMD        :1;
    unsigned          CSOpenCMD         :1;
    unsigned          CSCloseCMD        :1;
    unsigned          PDETLPOWER        :1;
```

```
    unsigned          PDETLTRN          :1; // (1 byte)

    unsigned          PDOPB             :1;
    unsigned          DCMLnotNormal     :1;
    unsigned          Emergency         :1;
    unsigned          Unused            :1;
    unsigned          DoorCloseStatus   :1;
    unsigned          DoorLockStatus    :1;
    unsigned          DSRLow            :1;
    unsigned          DSRHigh           :1; // (1 byte)

    unsigned          DCUMajorFail      :1;
    unsigned          DCUMinorFail      :1;
    unsigned          ReduceDoorSpeed   :1;
    unsigned          DisableOpen       :1;
    unsigned          DisableClose      :1;
    unsigned          Event13           :1;
    unsigned          Event15           :1;
    unsigned          Event17           :1; // (1 byte)

    unsigned          Event18           :1;
    unsigned          Event19           :1;
    unsigned          Event20           :1;
    unsigned          Event21           :1;
    unsigned          Event30           :1;
    unsigned          Event38           :1;
    unsigned          Event51           :1;
    unsigned          Event52           :1; // (1 byte)

    unsigned          Event53           :1;
    unsigned          Event65           :1;
    unsigned          Event66           :1;
    unsigned          Event68           :1;
    unsigned          Event69           :1;
    unsigned          Event71           :1;
    unsigned          Event72           :1;
    unsigned          Event73           :1; // (1 byte)

    unsigned          Event74           :1;
    unsigned          Event75           :1;
    unsigned          Event76           :1;
    unsigned          Event77           :1;
    unsigned          Event78           :1;
    unsigned          Event79           :1;
    unsigned          Event80           :1;
    unsigned          Event83           :1; // (1 byte )

    unsigned          uSpare1;          // (1 byte)
} struct_DCU_input_signals;             // Total of 7 bytes
```

The following Union will be used to efficiently update this structure when the information is received from DCU Application:

```
union union_DCU_input_signals
{
    struct_DCU_input_signals Separate;
    unsigned char       ucArray[7];//7 bytes match the 7 bytes
                                          in 'Separate' structure
};
```

3.5.2.5 SD3.20 Network Health Data

For easy access to the status of the network's health (information that is needed by many modules of NGA), the following structure will be used to maintain this information:

```
far struct
{
    boolean bNLC;             //No network communication for 60 seconds
    boolean bLonWorksErr;     //Based on monitoring of Firmware Errors
} NetworkHealth;
```

3.5.2.6 SD3.21 SPI Health Data

For easy access to the status of the SPI interface health (information that is needed by many modules of NGA), the following structure will be used to maintain this information:

```
typedef struct
{
    boolean bSpiError;
    boolean bBadSPI;
    unsigned char ucSPISyncErrors;
    unsigned char ucSPIChecksumErrors;
} struct_spi_health;

far struct_spi_health SpiHealth;
```

3.5.2.7 SD3.22 Location Data

For easy access to the DCU Location information (information that is needed by many modules of NGA), the following structure will be used to maintain this information:

```
typedef struct
{
    unsigned char PositionOutOf4; //1 of 4 positions
    boolean       InAcar;          //Is the DCU in A car
    unsigned char NodeID;          //Node ID defined in SRS
    unsigned char GroupMemberID;   //Group Member ID defined in SRS
} location_type;

location_type Location;
```

3.5.2.8 SD3.23 SCI CRC Data

For easy access to the CRCs of all of the Software Configuration Items within the DCU (information that is needed by more than one module of NGA), the following structure will be used to maintain this information:

```
unsigned long ulSCIsCRCArray[NUMBER_OF_SCIs]; //NUMBER_OF_SCIs is 5
```

3.5.2.9 SD3.26 FTP Status Data

To help manage the FTP process, the following structure will be used:

```
typedef struct
{
   unsigned      uCurrentWindow  :4;   //Window number; modulo 16.
   unsigned      uSpare          :1;
   unsigned      uPacketInWindow :3;   //Packet # within the Window.

} ftp_data_header_type;

//Structure to hold the complete status of the FTP process
typedef struct
{
   boolean                 bFtpOn;                    //Are we in FTP mode
   unsigned char           ucXferStatus;              //Off/To Start/Started/Done/Awaiting Close
   unsigned char           ucFileStatus;              //Status to be sent in nvoFileStatD NV
   unsigned long           ulBytesLeftForCopying;     //Remaining # of bytes after this window
   unsigned char          *ptrCurrentDataPointer;     //Point up to which data has been copied
   unsigned char          *ptrWrapAroundPoint;        //End of file in memory
   ftp_data_header_type   Header;
   unsigned char           ucBytesInPacket[PACKETS_PER_WINDOW];
                                                      //Number of bytes in the packets in
                                                      // the current window

//The following are for the parameters to be used for sending the Request/Response packets
// during the actual data/file transfer.
   unsigned              tx_timer     :4;
   unsigned              retry        :4;

   snode_struct          CDT_SendingInfo;

} ftp_status_type;
ftp_status_type          ftpStatus;
```

4 OPERATING SYSTEM SERVICES

The Neuron Gateway Application requires the following services from the Neuron Firmware (firmware required by Echelon for FT3150):

- Task Schedule
- Timer
- Send and Receive the Network Variable/Explicit Message
- Neuron Built In Function
- Neuron I/O Object
- Neuron Diagnostic

5 ERROR / WARNING HANDLING

The handled errors refer to the communication between the Neuron Gateway Application and the Car Network, the communication between the Neuron Gateway Application and the DCU Application, and the Neuron Self Test Failures.

All communication errors between Neuron Gateway Application and the Car Network are implemented in Neuron Firmware. The error related events are handled by the Neuron Gateway Application:

- nv_update_fails
- msg_fails

The communication between the Neuron Gateway Application and the DCU Application is implemented in Neuron Firmware as well. The communication packet integrity is controlled by a Checksum. If a Checksum error occurs, the packet is ignored.

If the error occurs during the Self Test, the Neuron Gateway Application logs it.

The Neuron Gateway Application may encounter errors/warnings that are listed in the following table.

Table 5-1: Faults List

#	Fault Name	Fault Description
1.	SRAM error	SRAM Memory Failure, detected by the Neuron Firmware.
2.	Application CRC error	Flash Memory Failure, detected by the Self Test.
3.	FT-X2 communication error	LonWorks Communication Error, detected by the Neuron Firmware
4.	Internal Register and port error	Neuron CPU Error, detected by the Neuron Firmware
5.	Watchdog Time error	Application Error, detected by the Neuron Firmware
6.	SPI communication error	SPI Communication CRC error, detected by the Application
7.	Invalid Location ID	The Location Identifier (connector strapping) incorrect
8.	Location Ident Circuit Fault	DCU's internal location ID circuit failure
9.	Self Installation Failure	Unable to set Neuron's Domain table as part of Self Installation

6 MEMORY LAYOUT

The Neuron FT3150 CPU memory space goes from addresses 0x0000 to 0xFFFF. The Neuron memory space is split into several sections with different purposes:

0x0000H~0x8FFFH: This section is designed as flash memory. The Neuron Firmware (firmware required by Echelon for FT3150) and DCU Neuron Gateway Application (binary) are stored in this area. 0x0000H~0x3FFFH (16K) is reserved for Neuron Firmware. 0x4000H~0x8FFFH (20K) is used by application.

0x9000H~0xAFFFH: This section is SRAM and it is used to store the Neuron Gateway Application variables.

0xB000H~0xDFFFH: This section works as NVSRAM. The data stored in this area is protected when the power is lost.

0xE800H~0xFFFFH: This memory segment resides in the Neuron FT3150 CPU. It is reserved for the Neuron FT3150 system. See Echelon document 005-0139-01B, FT 3120/ FT3150 Smart Transceiver Data Book for detail.

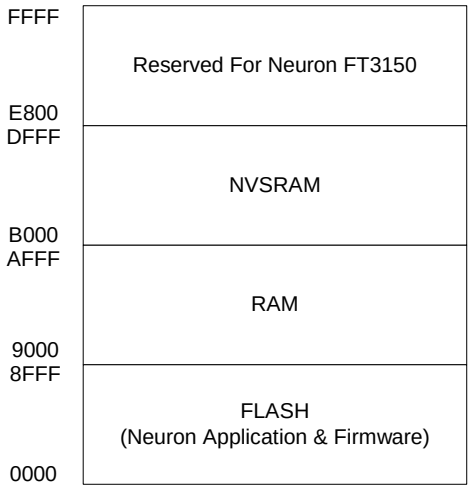

Figure 6-1: DCU Neuron Memory Map